

GATEWEB · PROJECT TALK TRACK

專案表達稿

關網資訊 AI 應用工程師面談用：三個專案怎麼講、被迫問怎麼接

依據：履歷（邱冠文 26 版）+ Threat Intel Agent 本機程式碼與 README

【請補：…】的地方只有你知道實際細節，請用真實情況填上，不要照抄

怎麼用這份稿

三種長度，依場合挑

長度	什麼時候用
一句話	自我介紹時帶過、HR 問「做過什麼」
30 秒版	面試官說「簡單介紹一下這個專案」
2 分鐘版	面試官說「詳細說說看」，或主動請你挑一個最有代表性的專案
追問卡	被問「為什麼這樣設計」「遇過什麼問題」時，挑對應的卡片回答

三個原則

1. **不要背稿**：記住每段的「重點順序」和關鍵數字，用自己的話講。照稿念，面試官一聽就知道。
2. **講「為什麼」比講「用了什麼」重要**：面試官想確認你懂自己做的東西。每個技術選擇都要能說出理由和取捨。
3. **不誇大，誠實說界線**：沒做過的就說沒做過，接著說你理解的做法。被抓到誇大，比承認不會傷害更大。

面談前一定要先處理的事

重要

Threat Intel Agent 的 GitHub 版本和你要講的內容不一致。

- GitHub main 分支 (8/11) 與線上 Render 網站：舊版「兩輪架構」(第一輪選工具、第二輪強制交報告)。
- 本機 fix/hardening 分支 (8/20)：新版「真正的 Tool Use 迴圈、全面非同步、安全修正、70 個測試」，沒有推上 GitHub。

面試官很可能會打開你的 GitHub。建議二選一：

1. 把 **fix/hardening 分支推上 GitHub** (只推分支、不合併 main，不會影響線上網站)，面談時可以說「改版在這個分支」。
2. **不推的話**，面談時就以「兩輪架構是第一版，之後我發現它其實可以被固定流程取代，第二版改成真正的迴圈，還在分支上」這個說法為準，不要說得像已經上線。

另外，第二版的改動你要能自己講清楚每一項的原因 (下面追問卡都有)，講不出來的就不要主動提。

用語對照：避免誇大

不要說	改成
「10 萬人搶票零超賣」	「用壓測腳本模擬 10~600 人同時搶票，確認零超賣」

不要說	改成
「精通 RAG、MCP、LangGraph」	「理解原理與流程，還沒有上線專案，正在補實作」
「串接過各家 LLM」	「實際串接的是 Anthropic Claude；OpenAI、Gemini 知道格式差異」
「系統準確率很高」	「判斷門檻寫死在規則裡、temperature 設 0 提高一致性，但還沒用標註測試集量化準確率」
「全部自己寫的」	依實際情況說明你負責的部分與 AI 工具輔助的部分（見 PART 5）

1 分鐘自我介紹（把三個專案串起來）

面試官好，我是邱冠文，明新科技大學資訊工程系今年畢業。

我在當兵期間擔任新訓中心的教育班長四年多，負責規劃訓練課程、帶團隊。退伍後回到資工系，並花了一年半專心做後端專案。

我做過三個比較完整的專案，剛好是一條往 AI 應用走的路：

第一個是**演唱會搶票系統**，用 FastAPI 和 Redis 處理高併發搶票，重點是「不能超賣」；

第二個是**四人團隊的電商後端**，我擔任隊長，負責架構切分和分工，也串了綠界的正式金流；

第三個是 **Threat Intel Agent**，用 Claude 的 Tool Use 讓 AI 自己決定要查哪些資安情資來源，最後產出結構化的風險報告，並用 Docker 部署上線。

關網的電子發票系統需要高併發、零錯誤、重資安，現在又要把 AI Agent、RAG 落地到實際業務，這跟我這三個專案累積的方向很接近，所以我很想加入。

重點順序：畢業背景 → 教育班長（團隊與溝通）→ 三個專案一句話 → 連到關網。

時間控制：60～80 秒。面試官有興趣會自己挑一個追問。

Threat Intel Agent (資安威脅情資 AI Agent)

一句話

輸入一個 IP 或網域，AI Agent 自己決定要查哪些資安情資來源，交叉比對後產出結構化的風險報告，高風險目標會自動列入黑名單，給防火牆查詢。

30 秒版

中小企業遇到可疑 IP，通常要手動查 WHOIS、VirusTotal、AbuseIPDB，再自己判斷，很花時間。

我做了一個 AI Agent：讓 Claude 自己選擇要查哪些來源，工具結果回饋給它再決定下一步，最後**強制它用固定格式交報告**，保證輸出一定能被程式解析。

系統會記錄 Agent 每一步查了什麼、花多久，可以回頭檢查判斷依據。後端是 FastAPI、資料庫是 MongoDB，用 Docker 部署在 Render。

2 分鐘版 (依「問題 → 做法 → 關鍵決策 → 結果 → 還沒做好的」)

問題：資安查證要比對好幾個來源，格式都不一樣，人工查一次要十幾分鐘，還需要專業判讀。

做法：我沒有用 LangChain，而是直接用 Anthropic SDK 寫 Tool Use。我定義了五個查詢工具（WHOIS、IPWHOIS、AbuseIPDB、VirusTotal 查 IP 與查網域），加上一個 `submit_report` 報告工具。

三個關鍵決策：

第一，**結構化輸出**。一開始用 prompt 要求模型輸出 JSON，還是常常解析失敗，所以我改成用 `tool_choice` 強制它呼叫 `submit_report`，欄位用 schema 定義，輸出一定可以解析。

第二，**外部 API 額度很少**，VirusTotal 免費版每分鐘只有 4 次，所以我寫了兩層裝飾器，把快取和計時包起來，五個工具共用，24 小時內重複查詢直接命中快取。

第三，**判斷和執行分開**。系統只負責判斷並寫入黑名單，提供查詢 API 給防火牆，不直接去封鎖，避免 AI 誤判就造成服務中斷。

結果：部署在 Render，有公開網址和 API 文件；每次分析的決策軌跡都存在資料庫，可以回溯「為什麼判定高風險」。

還沒做好的：我還沒有建立標註好的測試集，所以準確率和一致性還沒量化，這是我下一步最想補的。

版本演進 (講這段最能展現你懂 Agent)

注意 第二版在本機分支，請先依 PART 0 的建議處理 GitHub，再決定要不要主動講。

第一版是寫死的兩次呼叫：第一輪選工具查資料，第二輪強制交報告。

後來我發現一個問題：工具的結果從來沒有回饋給模型再決定下一步。這樣其實跟「查完所有來源、丟給 LLM 總結」的固定流程沒有差別，卻多付了 Agent 的成本（token 加倍、延遲、不確定性）。

所以第二版改成**真正的迴圈**：每一輪把工具結果放回對話，模型依結果決定要不要再查，最多 6 輪；**最後一輪強制交報告**，保證一定會結束。同一輪的多個查詢用 `asyncio.gather` 併發執行。

我也誠實記錄了限制：目前五個工具彼此獨立，模型其實很少需要第二輪以後的推理。要讓迴圈真正有價值，需要加入能互相牽引的工具，例如 IP 反查網域。

追問卡

Q1：為什麼不用 LangChain？

這個專案流程不複雜，我想**完整掌握每一步送了什麼給模型、模型回了什麼**，方便除錯，所以直接用 SDK。

我也理解框架的價值：流程有很多分支、需要記憶、檢索和追蹤時，LangChain 或 LangGraph 能省很多工。職缺提到要用 LangGraph 落地，這是我會補起來的部分。

Q2：為什麼要強制呼叫 `submit_report`，不直接叫它輸出 JSON？

模型輸出是機率性的，只靠 prompt 還是會偶爾輸出 Markdown 表格或缺欄位。用工具呼叫時，參數必須符合我定義的 schema（風險等級是固定幾個選項、證據是陣列），程式不需要寫一堆容錯解析。這也是「讓 LLM 輸出穩定可用」最實用的技巧。

Q3：AI 的風險判斷穩定嗎？（第二版內容）

一開始我只給模型四個風險等級的文字描述，讓它自己判斷，結果**同一個 IP、同一份快取資料，跑出過 HIGH 和 MEDIUM 兩種結論**。

所以我把判斷門檻寫死在規則裡，例如 AbuseIPDB 分數 75 以上、或 VirusTotal 惡意判定 5 個以上才算 HIGH，再把 temperature 設成 0。**標準由人訂，證據的蒐集與解讀交給模型**。

但我沒有量化改善了多少，下一步是建立 30 ~ 50 個標註好的目標當測試集。

Q4：快取怎麼設計？遇過什麼問題？

兩層裝飾器：內層 `@iscache` 查快取、沒命中才真的打 API 並寫回；外層 `@duration` 計時。Key 用「工具名稱 + 查詢目標」，避免不同工具互相覆蓋，TTL 24 小時。

第二版修了兩個坑【請確認你能解釋】：

- 原本用 `if cache:` 判斷命中，**快取值是空的 dict 時會被當成沒命中**，每次都重打 API。改成明確回傳「是否命中」。
- **錯誤回應不能寫進快取**：如果把一次 429 寫進去，這個目標之後 24 小時都只會拿到錯誤。

Q5：為什麼選 MongoDB？

各情資來源回傳的欄位差很多，報告裡的證據也是巢狀、不固定的結構，文件型資料庫比較自然，不用為了固定 schema 拆很多張表。

如果資料是發票、客戶這種結構固定、需要交易一致性的，我會選 PostgreSQL。

Q6：為什麼判斷和執行要分開？

AI 一定會誤判。如果系統判定高風險就直接封鎖，一次誤判就可能把正常客戶擋掉。所以系統只判斷、寫入黑名單，並提供 `GET /blocklist/check/{target}` 讓防火牆自己決定。

同樣的原則放到關網：AI 可以協助判斷發票異常，但開立、作廢這種動作應該要有人確認。

Q7：整個服務怎麼處理非同步？（第二版內容，對應職缺的 Asyncio）

一次分析要 10~30 秒。如果中間有同步阻塞的呼叫，例如 `requests` 或同步的 `whois` 查詢，單一 worker 會整個卡住，連健康檢查都回不了。

所以資料庫用 `pymongo` 的非同步客戶端、外部 API 用 `httpx.AsyncClient`、同步的 `whois` 函式庫用 `asyncio.to_thread` 丟到執行緒，同一輪的查詢用 `asyncio.gather` 併發。

Q8：安全上做了什麼？（第二版內容，對應筆試「測試安全」）

我自己找到兩個問題並修正【請確認你能解釋】：

- 第一版把 API Key 寫死在前端 HTML，任何人看原始碼就拿得到。後來改成：分析端點用**伺服器端的用量限制**保護（每個 IP 每小時 5 次、全站每日 200 次），管理用的金鑰永遠不下發到前端。
- LLM 產生的摘要會顯示在網頁上，如果工具回傳的內容被植入惡意指令，可能一路變成 XSS。所以所有插入畫面的內容都先跳脫，`system prompt` 也要求把工具回傳內容當成資料、不是指令。

Q9：怎麼測試一個會呼叫 LLM 的系統？（第二版內容）

測試不連資料庫、不打外部 API、不呼叫真的 LLM：資料庫換成記憶體假物件，Claude client 和工具也換成假實作。

測的東西包括：輸入驗證、快取命中與過期、Agent 迴圈有沒有真的把結果回饋、工具出錯時會不會正常收斂、限流和金鑰保護。目前 70 個測試全部通過。

Q10：如果重做或繼續做，你會改什麼？

- 1. 先建立**標註測試集**，量化準確率、同一目標重跑的一致性和每次的 token 成本。
- 2. 加入能**互相牽引的工具**（IP 反查網域、同網段已知惡意目標），讓多輪推理真的有價值。
- 3. 如果流程再變複雜，考慮用 **LangGraph** 管理狀態與人工審核節點。

對應關網職缺

關網職缺寫的	這個專案裡的對應
協助開發維護 Tool Calling，串接第三方 API	5 個外部情資 API，由模型決定呼叫；強制結構化輸出
LLM API 對接、測試與異常處理	外部 API 速率限制與快取；錯誤不寫入快取；工具出錯時的收斂
Agent 規劃機制	模型依工具結果決定下一步、最多 6 輪、最後一輪強制收斂
自動化測試（LLM 輸出品質、API）	不依賴外部服務的測試套件；還沒做的標註測試集是我的下一步
Python（FastAPI）微服務、Docker	FastAPI 分模組路由、Docker 部署
加分：Asyncio、資安或系統監控	全鏈路非同步；資安情資主題；用量限制、輸出跳脫

演唱會搶票系統（比賽作品暨畢業專題）

一句話

兩人協作的搶票系統，我負責全部後端，用 FastAPI 和 Redis 分散式鎖處理大量使用者同時搶同一個座位，確保不會超賣。

30 秒版

搶票最大的問題是很多人同時搶同一個位子。我用 Redis 的 SET NX 做分散式鎖，同一個座位只有第一個成功拿到鎖的人能進入下一步。

購票分成三段：先鎖位、再做 Google Authenticator 的 TOTP 雙重驗證、最後確認購買，也順便防機器人。

我寫了壓測腳本模擬 10 ~ 600 人同時搶票，確認零超賣。

2 分鐘版

問題：熱門場次開賣時，同一瞬間可能很多請求搶同一個座位。如果只是「先查這個位子空不空、再寫入」，兩個請求可能同時查到空位，結果兩個人都買到。

做法：

- 後端用 FastAPI 加 Uvicorn 非同步處理請求。
- 座位鎖定用 Redis 的 SET key value NX（只在 key 不存在時才設定成功），這個動作在 Redis 裡是原子性的，所以同一個座位同時只有一個人拿得到鎖。【請補：鎖有沒有設定過期時間、過期多久、為什麼】
- 購票流程分成鎖位、TOTP 驗證、確認三段，也有會員系統與 Session 驗證。

最有價值的經驗：壓測時我發現一個只在多程序併發下才會出現的 race condition，單機測試完全抓不到。【請補：實際的現象、根本原因、怎麼修】我走完了發現、找根因、修補、再跑一次壓測驗證的完整流程。

結果：壓測 10 ~ 600 人同時搶票，確認零超賣。

追問卡

Q1：為什麼用 Redis 鎖，不用資料庫的交易或鎖？

【請依你當時的理由調整】搶票是很短時間內大量讀寫同一批座位，Redis 在記憶體裡運作、速度快，SET NX 本身是原子操作，適合做「搶到就先佔住」的短暫鎖定；最後確認購買時，才把結果寫進 MySQL 保存。

如果用資料庫的列鎖，高併發時容易讓資料庫變成瓶頸。

Q2：那個 race condition 是怎麼找到的？

【請補：用什麼壓測工具或腳本、看到什麼數據異常、怎麼縮小範圍、最後原因與修法】

回答結構建議：現象（壓測出現什麼）→ 為什麼單程序抓不到 → 根因 → 修法 → 怎麼驗證修好了。

Q3：如果使用者鎖了位子却不付款怎麼辦？

【請依實作回答】常見做法是鎖設定過期時間，時間到自動釋放，別人就能再搶。

Q4：跟關網有什麼關係？

電子發票系統每個月處理幾千萬張發票，同樣是「高併發、不能出錯」。像開立發票重試時不能重複開立，就跟搶票不能超賣是同一類問題：檢查和寫入之間不能有空隙。

MineMarket 電商平台（四人團隊，擔任隊長）

一句話

四人協作的遊戲道具電商後端，我當隊長，負責時程、會議、架構切分與分工，自己開發認證模組，也整合了綠界正式金流和遊戲伺服器即時發放道具。

30 秒版

這是四個人一起做的電商後端。我擔任隊長，用 Flask Blueprint 把系統切成認證、商品、購物車、後台四個模組，讓大家可以平行開發、減少合併衝突。

我負責認證模組，也寫了幾個全隊共用的 decorator。

金流串接綠界的正式環境，用 CheckMacValue 簽章驗證；付款成功後透過 Minecraft 的 RCON 即時發放道具，玩家不在線時由背景任務排隊補發。

2 分鐘版

團隊面：四個人程度不一，最大的風險是大家改到同一份檔案、互相覆蓋。所以我先規劃模組邊界，用 Blueprint 切成四塊，每個人負責一塊，共用的東西（路由守衛、參數解析、資料庫交易）由我寫成 decorator 給大家用。我也負責排時程、主持會議。

技術面：

- **認證模組：**會員註冊（含 Email 驗證、Cloudinary 上傳大頭貼）、登入、Session 驗證，並在資料庫層做單一裝置登入。
- **Decorator：**其中一個用 inspect 讀取函式參數，自動從請求組好參數傳進去，減少每支 API 重複寫解析。
- **金流：**串綠界正式金流，用 CheckMacValue 驗證通知確實來自綠界、內容沒被竄改。
- **即時發放：**訂單成立後透過 RCON 下指令給遊戲伺服器發道具；玩家離線時進入背景排隊，上線後補發。
- **部署：**Heroku 加 gunicorn。圖片放 Cloudinary，因為 Heroku 檔案系統重啟會清空；遊戲伺服器在本機，用 Ngrok 反向代理讓雲端連得到。

學到的：架構設計不只是技術決策，也要考慮團隊分工的邊界。

追問卡

Q1：CheckMacValue 是什麼？為什麼一定要驗證？

綠界付款完成後會通知我們的伺服器。任何人都可以偽造一個「付款成功」的請求打到這個網址，所以要用雙方約定的金鑰，依規則對參數算出雜湊簽章，跟通知裡的 CheckMacValue 比對，一致才代表真的是綠界送的、內容也沒被改。

這跟關網的電子發票 API 串接、快收慢付金流是同一類概念：**對外介面一定要驗證來源與完整性。**

Q2：玩家離線時的補發怎麼做？

【請依實作補充：佇列存在哪裡、多久檢查一次、怎麼避免重複發放】

重點要講到**幂等性**：同一筆訂單不管重試幾次，道具只能發一次。

Q3：當隊長時遇過意見不合或衝突嗎？（也對應第三關情境題）

【請用真實事件，照 STAR 結構】情境：… 任務：… 你做了什麼：先聽對方理由 → 回到共同目標（例如準時交付、減少衝突）→ 用具體方式決定（小規模試做、比較成本）→ 決定後全力執行。 結果與學到的：…

Q4：為什麼用 Blueprint？

Blueprint 讓每個功能模組有自己的路由與邏輯，檔案分開，四個人能平行開發、減少 Git 合併衝突，也比較好找問題。

一定會遇到的通用追問

「這些程式碼是你自己寫的，還是 AI 寫的？」

請依真實情況回答 這題最怕說謊。面試官問的通常不是「有沒有用 AI」，而是「你懂不懂」。

參考結構（請改成符合你實際做法的版本）：

我有使用 Claude Code 這類 AI 工具輔助開發，業界現在也很普遍。

但【請選擇真實的部分：需求怎麼拆、架構怎麼選、工具怎麼定義、遇到 bug 怎麼查、怎麼驗證】是我主導的。像 Threat Intel Agent 從兩輪改成迴圈，就是我發現「工具結果沒有回饋，其實等於固定流程」之後決定改的。

每一個設計決策，我都可以說明為什麼這樣做、還有哪些沒做好。

要做到：上面追問卡的內容你都能用自己的話講出來。講不出來的部分，就不要宣稱是自己主導的。

「你實際負責哪些部分？」

專案	你的角色
搶票系統	兩人協作，負責全部後端架構
MineMarket	四人團隊隊長，負責時程、會議、架構切分、認證模組、共用 decorator、部署
Threat Intel Agent	個人專案，從規劃、實作到部署

「遇過最難的 bug 是什麼？」

挑一個，用「現象 → 為什麼難找 → 怎麼縮小範圍 → 根因 → 修法 → 怎麼驗證」講：

- 首選：搶票系統只在多程序併發才出現的 race condition【請補細節】。
- 備選：Threat Intel Agent 快取值是空 dict 時被當成沒命中，導致最該省額度的時候反而一直重打 API。

「你沒做過 RAG，進來怎麼辦？」

我還沒有 RAG 的上線專案，這是實話。

但我理解完整流程：文件解析、清洗、切塊、embedding、存進向量資料庫，查詢時做相似度搜尋，必要時混合關鍵字檢索和 rerank，再把段落放進 prompt 要求依據回答。答錯時要先分辨是檢索錯還是生成錯。

我在 Threat Intel Agent 做過類似的事：把外部資料整理成結構化證據交給模型判斷。進來後我希望先從維護現有的 RAG pipeline 和建立測試集開始。

「為什麼只串過 Claude？」

專案只需要一家，所以我選了 Claude。我知道各家格式有差異，例如 Claude 的 system 是獨立參數、工具呼叫用 tool_use 和 tool_result 內容區塊，OpenAI 則用 tool_calls 和 role 為 tool 的訊息。

要同時支援多家時，我會定義一個共同介面，每家寫一個轉接器，並統一錯誤處理和備援切換。

「你的缺點是什麼？」

我實作經驗走得比理論快。很多東西我先做出來，後來才回頭把原理補完整，例如 Transformer、RAG 的調校細節。

我現在的做法是：做完一個功能，就回頭整理「為什麼這樣設計、有哪些替代方案」，像 Threat Intel Agent 的 README 就記錄了每個設計決策和已知限制。